

btt006__unit2__lab-solution

April 20, 2026

1 Lab 2: ML Life Cycle: Data Understanding and Data Preparation

```
[1]: import os
import pandas as pd
import numpy as np
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns
```

1.1 Business Brief

Read through the scenario below. You will be putting yourself in the shoes of a junior ML engineer (MLE) at NestIQ, a company tasked with developing a machine learning model to predict the prices of new short-term rentals for one of its clients.

1.1.1 Company & Context

NestIQ is a data and analytics company that develops machine learning models to help real estate and short-term rental clients make better pricing decisions.

1.1.2 Business Challenge

NestIQ's new client is a short-term rental operator based in New York City (NYC). They currently set prices manually or by using simple rules. For example, they start with a base price and increase it by a fixed amount for each additional bedroom, adjust for room type, and apply small premiums for highly-rated hosts and location. These rules provide a starting point, but pricing is often adjusted manually on a case-by-case basis. As the company grows, manual pricing becomes difficult to scale.

1.1.3 Business Goal

NestIQ's goal is to develop a pricing model for the client that estimates nightly listing prices for new short-term rentals. The supervised machine learning model should learn relationships

between listing features and prices from the client’s current NYC short-term rental listings data and accurately predict the price of a new listing.

1.1.4 Your Role and Task

You have just joined NestIQ as a junior MLE on the Pricing Model team. The team has been given a dataset of the operator’s current NYC short-term rental listings. It will use this data to train the pricing model.

Your task is to explore the data and prepare it for modeling. The choices you make during this process — such as how to handle missing values and outliers — will affect the information available to the model during training and ultimately its performance.

This work involves making decisions about the data, for example:

- Dropping columns with many missing values might remove useful information.
- Modifying outliers may alter meaningful patterns in the data.

1.1.5 Technical Focus in This Lab

In this lab, you will practice the second and third steps of the machine learning life cycle: data understanding and data preparation. You will begin preparing your data so that it can be used to train a machine learning model that solves a regression problem. Note that by the end of the lab, your data set won’t be completely ready for the modeling phase, but you will gain experience using some common data preparation techniques.

You will use the Airbnb “listings” data set to represent the type of data NestIQ would receive from its short-term rental client.

You will complete the following tasks to transform your data:

1. Build your data matrix and define your ML problem:
 - Load the Airbnb “listings” data set into a DataFrame and inspect the data
 - Define the label and convert the label’s data type to one that is more suitable for modeling
 - Identify features
2. Clean your data:
 - Handle outliers by building a new regression label column by winsorizing outliers
 - Handle missing data by replacing all missing values in the dataset with means
3. Perform feature transformation using one-hot encoding
4. Explore your data:
 - Identify two features with the highest correlation with label
 - Build appropriate bivariate plots to visualize the correlations between features and the label
5. Analysis:
 - Analyze the relationship between the features and the label
 - Brainstorm what else needs to be done to fully prepare the data for modeling

1.2 Part 1. Build Your Data Matrix (DataFrame) and Define Your ML Problem

We will be working with the Airbnb NYC "listings" data set. Use the specified path and name of the file to load the data. Save it as a Pandas DataFrame called `df`.

```
[2]: # Do not remove or edit the line below:
filename = os.path.join(os.getcwd(), "data", "airbnbData.csv")
```

Task: Load the data and save it to DataFrame `df`.

Note: You may receive a warning message. Ignore this warning.

```
[3]: # YOUR CODE HERE

# solution
df = pd.read_csv(filename)
```

```
/usr/local/lib/python3.6/dist-packages/IPython/core/interactiveshell.py:2728:
DtypeWarning: Columns (67) have mixed types.Specify dtype option on import or
set low_memory=False.
```

```
interactivity=interactivity, compiler=compiler, result=result)
```

Task: Display the shape of `df` -- that is, the number of rows and columns.

```
[4]: # YOUR CODE HERE

# Solution
df.shape
```

```
[4]: (38277, 74)
```

Task: Display the column names.

```
[5]: # YOUR CODE HERE

# solution
df.columns
```

```
[5]: Index(['id', 'listing_url', 'scrape_id', 'last_scraped', 'name', 'description',
'neighborhood_overview', 'picture_url', 'host_id', 'host_url',
'host_name', 'host_since', 'host_location', 'host_about',
'host_response_time', 'host_response_rate', 'host_acceptance_rate',
'host_is_superhost', 'host_thumbnail_url', 'host_picture_url',
'host_neighbourhood', 'host_listings_count',
'host_total_listings_count', 'host_verifications',
'host_has_profile_pic', 'host_identity_verified', 'neighbourhood',
'neighbourhood_cleansed', 'neighbourhood_group_cleansed', 'latitude',
'longitude', 'property_type', 'room_type', 'accommodates', 'bathrooms',
'bathrooms_text', 'bedrooms', 'beds', 'amenities', 'price',
'minimum_nights', 'maximum_nights', 'minimum_minimum_nights',
```

```

'maximum_minimum_nights', 'minimum_maximum_nights',
'maximum_maximum_nights', 'minimum_nights_avg_ntm',
'maximum_nights_avg_ntm', 'calendar_updated', 'has_availability',
'availability_30', 'availability_60', 'availability_90',
'availability_365', 'calendar_last_scraped', 'number_of_reviews',
'number_of_reviews_ltm', 'number_of_reviews_l30d', 'first_review',
'last_review', 'review_scores_rating', 'review_scores_accuracy',
'review_scores_cleanliness', 'review_scores_checkin',
'review_scores_communication', 'review_scores_location',
'review_scores_value', 'license', 'instant_bookable',
'calculated_host_listings_count',
'calculated_host_listings_count_entire_homes',
'calculated_host_listings_count_private_rooms',
'calculated_host_listings_count_shared_rooms', 'reviews_per_month'],
dtype='object')

```

Task: Get a peek at the data by displaying the first few rows, as you usually do.

```
[6]: # YOUR CODE HERE
```

```

# Solution
df.head()

```

```

[6]:      id      listing_url      scrape_id last_scraped \
0  2595  https://www.airbnb.com/rooms/2595  20211204143024  2021-12-05
1  3831  https://www.airbnb.com/rooms/3831  20211204143024  2021-12-05
2  5121  https://www.airbnb.com/rooms/5121  20211204143024  2021-12-05
3  5136  https://www.airbnb.com/rooms/5136  20211204143024  2021-12-05
4  5178  https://www.airbnb.com/rooms/5178  20211204143024  2021-12-05

      name \
0      Skylit Midtown Castle
1  Whole flr w/private bdrm, bath & kitchen(pls r...
2      BlissArtsSpace!
3      Spacious Brooklyn Duplex, Patio + Garden
4      Large Furnished Room Near B'way

      description \
0  Beautiful, spacious skylit studio in the heart...
1  Enjoy 500 s.f. top floor in 1899 brownstone, w...
2  <b>The space</b><br />HELLO EVERYONE AND THANK...
3  We welcome you to stay in our lovely 2 br dupl...
4  Please don't expect the luxury here just a bas...

      neighborhood_overview \
0  Centrally located in the heart of Manhattan ju...
1  Just the right mix of urban center and local n...

```

2	NaN
3	NaN
4	Theater district, many restaurants around here.

	picture_url	host_id	\
0	https://a0.muscache.com/pictures/f0813a11-40b2...	2845	
1	https://a0.muscache.com/pictures/e49999c2-9fd5...	4869	
2	https://a0.muscache.com/pictures/2090980c-b68e...	7356	
3	https://a0.muscache.com/pictures/miso/Hosting-...	7378	
4	https://a0.muscache.com/pictures/12065/f070997...	8967	

	host_url	...	review_scores_communication	\
0	https://www.airbnb.com/users/show/2845	...	4.79	
1	https://www.airbnb.com/users/show/4869	...	4.80	
2	https://www.airbnb.com/users/show/7356	...	4.91	
3	https://www.airbnb.com/users/show/7378	...	5.00	
4	https://www.airbnb.com/users/show/8967	...	4.42	

	review_scores_location	review_scores_value	license	instant_bookable	\
0	4.86	4.41	NaN	f	
1	4.71	4.64	NaN	f	
2	4.47	4.52	NaN	f	
3	4.50	5.00	NaN	f	
4	4.87	4.36	NaN	f	

	calculated_host_listings_count	calculated_host_listings_count_entire_homes	\
0	3	3	
1	1	1	
2	2	0	
3	1	1	
4	1	0	

	calculated_host_listings_count_private_rooms	\
0	0	
1	0	
2	2	
3	0	
4	1	

	calculated_host_listings_count_shared_rooms	reviews_per_month
0	0	0.33
1	0	4.86
2	0	0.52
3	0	0.02
4	0	3.68

[5 rows x 74 columns]

Define the Label Assume that your goal is to train a machine learning model that predicts the price of an Airbnb. This is an example of supervised learning and is a regression problem. In our dataset, our label will be the `price` column. Let's inspect the values in the `price` column.

```
[7]: df['price']
```

```
[7]: 0      $150.00
     1      $75.00
     2      $60.00
     3     $275.00
     4      $68.00
     ...
    38272    $79.00
    38273    $76.00
    38274   $116.00
    38275   $106.00
    38276   $689.00
     Name: price, Length: 38277, dtype: object
```

Notice the `price` column contains values that are listed as `<currency_name><numeric_value>`. For example, it contains values that look like this: `$120`.

Task: Obtain the data type of the values in this column:

```
[8]: # YOUR CODE HERE

     # Solution
     df['price'].dtype
```

```
[8]: dtype('O')
```

Notice that the data type is "object," which in Pandas translates to the String data type.

Task: Display the first 15 unique values of the `price` column:

```
[9]: # YOUR CODE HERE

     ### Solution:
     df['price'].unique()[:15]
```

```
[9]: array(['$150.00', '$75.00', '$60.00', '$275.00', '$68.00', '$98.00',
          '$89.00', '$65.00', '$62.00', '$90.00', '$199.00', '$96.00',
          '$299.00', '$140.00', '$175.00'], dtype=object)
```

In order for us to use the prices for modeling, we will have to transform the values in the `price` column from strings to floats. We will:

- remove the dollar signs (in this case, the platform forces the currency to be the USD, so we do not need to worry about targeting, say, the Japanese Yen sign, nor about converting the values into USD).

- remove the commas from all values that are in the thousands or above: for example, \$2,500.

The code cell below accomplishes this.

```
[10]: df['price'] = df['price'].str.replace(',', '')
df['price'] = df['price'].str.replace('$', '')
df['price'] = df['price'].astype(float)
```

Task: Display the first 15 unique values of the price column again to make sure they have been transformed.

```
[11]: # YOUR CODE HERE

### Solution:
df['price'].unique()[:15]
```

```
[11]: array([150., 75., 60., 275., 68., 98., 89., 65., 62., 90., 199.,
          96., 299., 140., 175.])
```

Identify Features Simply by inspecting the data, let's identify some columns that should not serve as features - those that will not help us solve our predictive ML problem.

Some that stand out are columns that contain website addresses (URLs).

Task: Create a list which contains the names of columns that contain URLs. Save the resulting list to variable `url_colnames`.

Tip: There are different ways to accomplish this, including using Python list comprehensions.

```
[12]: #url_colnames = # YOUR CODE HERE
#url_colnames

### Solution:
url_colnames = [x for x in list(df.columns) if '_url' in x]
url_colnames
```

```
[12]: ['listing_url',
       'picture_url',
       'host_url',
       'host_thumbnail_url',
       'host_picture_url']
```

Task: Drop the columns with the specified names contained in list `url_colnames` in place (that is, make sure this change applies to the original DataFrame `df`, instead of creating a temporary new DataFrame object with fewer columns).

```
[13]: # YOUR CODE HERE

### Solution:
df.drop(url_colnames, axis = 1, inplace=True)
```

Task: Display the shape of the data to verify that the new number of columns is what you expected.

```
[14]: # YOUR CODE HERE
```

```
### Solution:
```

```
df.shape
```

```
[14]: (38277, 69)
```

Task: In the code cell below, display the features that we will use to solve our ML problem.

```
[15]: # YOUR CODE HERE
```

```
### Solution:
```

```
list(df.loc[:, df.columns != 'Price'])
```

```
[15]: ['id',  
      'scrape_id',  
      'last_scraped',  
      'name',  
      'description',  
      'neighborhood_overview',  
      'host_id',  
      'host_name',  
      'host_since',  
      'host_location',  
      'host_about',  
      'host_response_time',  
      'host_response_rate',  
      'host_acceptance_rate',  
      'host_is_superhost',  
      'host_neighbourhood',  
      'host_listings_count',  
      'host_total_listings_count',  
      'host_verifications',  
      'host_has_profile_pic',  
      'host_identity_verified',  
      'neighbourhood',  
      'neighbourhood_cleansed',  
      'neighbourhood_group_cleansed',  
      'latitude',  
      'longitude',  
      'property_type',  
      'room_type',  
      'accommodates',  
      'bathrooms',  
      'bathrooms_text',
```



```
'bedrooms',
'beds',
'amenities',
'price',
'minimum_nights',
'maximum_nights',
'minimum_minimum_nights',
'maximum_minimum_nights',
'minimum_maximum_nights',
'maximum_maximum_nights',
'minimum_nights_avg_ntm',
'maximum_nights_avg_ntm',
'calendar_updated',
'has_availability',
'availability_30',
'availability_60',
'availability_90',
'availability_365',
'calendar_last_scraped',
'number_of_reviews',
'number_of_reviews_ltm',
'number_of_reviews_l30d',
'first_review',
'last_review',
'review_scores_rating',
'review_scores_accuracy',
'review_scores_cleanliness',
'review_scores_checkin',
'review_scores_communication',
'review_scores_location',
'review_scores_value',
'license',
'instant_bookable',
'calculated_host_listings_count',
'calculated_host_listings_count_entire_homes',
'calculated_host_listings_count_private_rooms',
'calculated_host_listings_count_shared_rooms',
'reviews_per_month']
```

Task: Are there any other features that you think may not be well suited for our machine learning problem? Note your findings in the markdown cell below.

Solution: Solutions may vary. Students can consider that features such as the name of the host and the location of the host do not add much value.

1.3 Part 2. Clean Your Data

Let's now handle outliers and missing data.

1.3.1 a. Handle Outliers

Let us prepare the data in our label column. Namely, we will detect and replace outliers in the data using winsorization.

Task: Create a new version of the `price` column, named `label_price`, in which you will replace the top and bottom 1% outlier values with the corresponding percentile value. Add this new column to the DataFrame `df`.

Remember, you will first need to load the `stats` module from the `scipy` package:

```
[16]: # YOUR CODE HERE

#### Solution:
import scipy.stats as stats
df['label_price'] = stats.mstats.winsorize(df['price'], limits=[0.01, 0.01])
```

Let's verify that the new column `label_price` was added to DataFrame `df`:

```
[17]: df.head()
```

```
[17]:
```

	id	scrape_id	last_scraped	\
0	2595	20211204143024	2021-12-05	
1	3831	20211204143024	2021-12-05	
2	5121	20211204143024	2021-12-05	
3	5136	20211204143024	2021-12-05	
4	5178	20211204143024	2021-12-05	

	name	\
0	Skylit Midtown Castle	
1	Whole flr w/private bdrm, bath & kitchen(pls r...	
2	BlissArtsSpace!	
3	Spacious Brooklyn Duplex, Patio + Garden	
4	Large Furnished Room Near B'way	

	description	\
0	Beautiful, spacious skylit studio in the heart...	
1	Enjoy 500 s.f. top floor in 1899 brownstone, w...	
2	The space HELLO EVERYONE AND THANK...	
3	We welcome you to stay in our lovely 2 br dupl...	
4	Please don't expect the luxury here just a bas...	

	neighborhood_overview	host_id	host_name	\
0	Centrally located in the heart of Manhattan ju...	2845	Jennifer	

```

1 Just the right mix of urban center and local n... 4869 LisaRoxanne
2                                     NaN      7356      Garon
3                                     NaN      7378      Rebecca
4 Theater district, many restaurants around here. 8967      Shunichi

  host_since      host_location ... review_scores_location \
0 2008-09-09 New York, New York, United States ...      4.86
1 2008-12-07 New York, New York, United States ...      4.71
2 2009-02-03 New York, New York, United States ...      4.47
3 2009-02-03 Brooklyn, New York, United States ...      4.50
4 2009-03-03 New York, New York, United States ...      4.87

  review_scores_value license instant_bookable calculated_host_listings_count \
0      4.41      NaN      f      3
1      4.64      NaN      f      1
2      4.52      NaN      f      2
3      5.00      NaN      f      1
4      4.36      NaN      f      1

  calculated_host_listings_count_entire_homes \
0      3
1      1
2      0
3      1
4      0

  calculated_host_listings_count_private_rooms \
0      0
1      0
2      2
3      0
4      1

  calculated_host_listings_count_shared_rooms reviews_per_month label_price
0      0      0.33      150.0
1      0      4.86      75.0
2      0      0.52      60.0
3      0      0.02      275.0
4      0      3.68      68.0

```

[5 rows x 70 columns]

Task: Check that the values of `price` and `label_price` are *not* identical.

You will do this by subtracting the two columns and finding the resulting *unique values* of the resulting difference. Note: If all values are identical, the difference would not contain unique values. If this is the case, outlier removal did not work.

```
[18]: # YOUR CODE HERE
```

```
### Solution:
```

```
(df['price']-df['label_price']).unique()
```

```
[18]: array([ 0.000e+00,  1.500e+03,  3.000e+02,  1.000e+03,  1.979e+03,
 -1.000e+00,  8.990e+02,  2.000e+02,  9.990e+02,  5.000e+02,
 -8.000e+00,  5.000e+03,  4.250e+03,  5.500e+02,  2.500e+02,
  5.500e+03,  1.750e+03,  2.750e+03,  6.000e+02, -1.100e+01,
  1.249e+03,  4.330e+02,  5.700e+01,  3.930e+02, -4.000e+00,
  4.000e+02,  1.695e+03,  8.990e+03,  2.140e+02, -1.400e+01,
  8.999e+03,  7.630e+02, -2.000e+00, -9.000e+00,  2.430e+02,
  1.000e+02,  6.400e+01,  2.974e+03,  7.700e+01, -3.000e+00,
 -7.000e+00,  3.500e+02,  2.450e+02,  8.100e+01,  5.710e+02,
  6.314e+03, -5.000e+00, -1.000e+01,  2.000e+00,  9.900e+01,
  1.200e+03,  4.300e+02,  1.100e+03,  8.500e+01,  4.000e+03,
  9.000e+03,  1.350e+03,  5.000e+01,  2.000e+03,  1.299e+03,
  1.430e+02,  1.499e+03,  3.700e+02, -1.900e+01,  6.184e+03,
 -1.300e+01,  2.210e+02,  1.857e+03, -1.500e+01,  9.000e+02,
  7.500e+01, -6.000e+00,  6.430e+02,  3.929e+03,  2.910e+02,
  3.990e+02,  8.000e+03,  5.429e+03,  3.000e+03, -1.800e+01,
  5.143e+03,  1.400e+03,  4.750e+02,  2.214e+03,  1.910e+02,
  4.250e+02,  1.250e+02,  3.330e+02,  4.990e+02,  8.000e+02,
  2.250e+02,  2.500e+03,  8.190e+02,  6.000e+03,  3.030e+02,
  3.070e+02,  1.640e+02,  3.420e+02,  5.600e+01,  2.600e+03,
  2.200e+03,  5.700e+02,  1.642e+03,  7.000e+00,  9.810e+02,
  2.120e+02,  1.850e+03,  4.500e+01,  4.510e+02,  5.120e+02,
  2.360e+02,  6.200e+01,  1.020e+02,  2.590e+02,  7.500e+02,
  9.750e+02,  5.290e+02,  2.960e+02,  9.500e+02,  1.600e+03,
  2.750e+02,  4.640e+02,  2.570e+02, -2.900e+01, -1.700e+01,
  9.500e+01,  2.850e+02,  3.382e+03,  1.839e+03,  1.261e+03,
  2.900e+01,  2.260e+02,  1.130e+02,  9.000e+00,  2.160e+02,
  1.160e+02, -1.200e+01,  4.950e+02,  2.500e+01,  2.860e+02,
  2.557e+03,  1.614e+03,  7.100e+01,  5.400e+01,  5.750e+02,
  1.700e+03,  2.400e+01,  1.700e+01,  1.140e+02,  2.900e+02,
  2.990e+02,  9.950e+02,  1.760e+02,  8.300e+02,  2.520e+03,
  8.650e+02,  6.700e+01,  1.797e+03,  2.729e+03,  7.600e+02,
  1.640e+03,  6.860e+02,  2.490e+02,  3.730e+02,  5.500e+01,
  7.420e+02,  2.920e+02,  1.436e+03,  3.860e+02,  3.570e+02,
  4.740e+02,  2.333e+03,  1.100e+01,  1.400e+01,  3.143e+03,
  4.500e+02,  8.300e+01,  1.990e+02,  8.560e+02,  1.370e+02,
  7.600e+01,  1.290e+02,  6.540e+02,  3.400e+01,  3.690e+02,
  8.170e+02,  4.790e+02,  8.970e+02,  3.140e+02,  3.320e+02,
  2.820e+02,  1.090e+02,  1.260e+02,  1.490e+02,  2.110e+02,
  1.232e+03,  3.464e+03,  2.119e+03,  3.310e+02,  5.650e+02,
  1.071e+03,  2.855e+03,  1.050e+03,  1.157e+03,  4.655e+03,
  9.800e+02])
```

1.3.2 b. Handle Missing Data

Next we are going to find missing values in our entire dataset and impute the missing values by replace them with means.

Identifying missingness **Task:** Check if a given value in the data is missing, and sum up the resulting values by columns. Save this sum to variable `nan_count`. Print the results.

```
[19]: #nan_count = # YOUR CODE HERE
      #nan_count

      ### Solution:
      nan_count = np.sum(df.isnull(), axis = 0)
      nan_count
```

```
[19]: id                                0
      scrape_id                          0
      last_scraped                        0
      name                               13
      description                        1192
      ...
      calculated_host_listings_count_entire_homes  0
      calculated_host_listings_count_private_rooms  0
      calculated_host_listings_count_shared_rooms  0
      reviews_per_month                      9504
      label_price                            0
      Length: 70, dtype: int64
```

Those are more columns than we can eyeball! For this exercise, we don't care about the number of missing values -- we just want to get a list of columns that have *any* missing values.

Task: From the variable `nan_count`, create a new series called `nan_detected` that contains `True` or `False` values that indicate whether the number of missing values is *not zero*:

```
[20]: #nan_detected = # YOUR CODE HERE
      #nan_detected

      ### Solution:
      nan_detected = nan_count!=0
      nan_detected
```

```
[20]: id                                False
      scrape_id                          False
      last_scraped                        False
      name                               True
      description                        True
      ...
```

```

calculated_host_listings_count_entire_homes    False
calculated_host_listings_count_private_rooms   False
calculated_host_listings_count_shared_rooms    False
reviews_per_month                             True
label_price                                    False
Length: 70, dtype: bool

```

Since replacing the missing values with the mean only makes sense for the columns that contain numerical values (and not for strings), let us create another condition: the *type* of the column must be int or float.

Task: Create a series that contains `True` if the type of the column is either `int64` or `float64`. Save the results to the variable `is_int_or_float`.

```

[21]: #is_int_or_float = # YOUR CODE HERE
      #is_int_or_float

      ### Solution:
      is_int_or_float = (df.dtypes == 'int64') | (df.dtypes == 'float64')
      is_int_or_float

```

```

[21]: id                                True
      scrape_id                          True
      last_scraped                       False
      name                               False
      description                        False

      ...
      calculated_host_listings_count_entire_homes    True
      calculated_host_listings_count_private_rooms   True
      calculated_host_listings_count_shared_rooms    True
      reviews_per_month                             True
      label_price                                    True
      Length: 70, dtype: bool

```

Task: Combine the two binary series (`nan_detected` and `is_int_or_float`) into a new series named `to_impute`. It will contain the value `True` if a column contains missing values *and* is of type 'int' or 'float'

```

[22]: #to_impute = # YOUR CODE HERE
      #to_impute

      ### Solution:
      to_impute = nan_detected & is_int_or_float
      to_impute

```

```

[22]: id                                False
      scrape_id                          False

```

```

last_scraped                False
name                        False
description                  False
...
calculated_host_listings_count_entire_homes    False
calculated_host_listings_count_private_rooms   False
calculated_host_listings_count_shared_rooms    False
reviews_per_month                          True
label_price                                False
Length: 70, dtype: bool

```

Finally, let's display a list that contains just the selected column names contained in `to_impute`:

```
[23]: df.columns[to_impute]
```

```
[23]: Index(['host_listings_count', 'host_total_listings_count', 'bathrooms',
            'bedrooms', 'beds', 'minimum_minimum_nights', 'maximum_minimum_nights',
            'minimum_maximum_nights', 'maximum_maximum_nights',
            'minimum_nights_avg_ntm', 'maximum_nights_avg_ntm', 'calendar_updated',
            'review_scores_rating', 'review_scores_accuracy',
            'review_scores_cleanliness', 'review_scores_checkin',
            'review_scores_communication', 'review_scores_location',
            'review_scores_value', 'reviews_per_month'],
           dtype='object')
```

We just identified and displayed the list of candidate columns for potentially replacing missing values with the column mean.

Assume that you have decided that you should impute the values for these specific columns: `host_listings_count`, `host_total_listings_count`, `bathrooms`, `bedrooms`, and `beds`:

```
[ ]: to_impute_selected = ['host_listings_count', 'host_total_listings_count',
    ↪ 'bathrooms',
    ↪ 'bedrooms', 'beds']
```

Keeping record of the missingness: creating dummy variables As a first step, you will now create dummy variables indicating the missingness of the values.

Task: For every column listed in `to_impute_selected`, create a new corresponding column called `<original-column-name>_na`. These columns should contain the a `True` or `False` value in place of `NaN`.

```
[ ]: ### YOUR CODE HERE

### Solution:
for colname in to_impute_selected:
    df[colname + '_na'] = df[colname].isnull()
```

Check that the DataFrame contains the new variables:

```
[ ]: df.head()
```

```
[ ]:      id      scrape_id last_scraped \
0  2595  20211204143024  2021-12-05
1  3831  20211204143024  2021-12-05
2  5121  20211204143024  2021-12-05
3  5136  20211204143024  2021-12-05
4  5178  20211204143024  2021-12-05

      name \
0      Skylit Midtown Castle
1  Whole flr w/private bdrm, bath & kitchen(pls r...
2      BlissArtsSpace!
3      Spacious Brooklyn Duplex, Patio + Garden
4      Large Furnished Room Near B'way

      description \
0  Beautiful, spacious skylit studio in the heart...
1  Enjoy 500 s.f. top floor in 1899 brownstone, w...
2  <b>The space</b><br />HELLO EVERYONE AND THANK...
3  We welcome you to stay in our lovely 2 br dupl...
4  Please don't expect the luxury here just a bas...

      neighborhood_overview  host_id  host_name \
0  Centrally located in the heart of Manhattan ju...  2845  Jennifer
1  Just the right mix of urban center and local n...  4869  LisaRoxanne
2      NaN  7356  Garon
3      NaN  7378  Rebecca
4  Theater district, many restaurants around here.  8967  Shunichi

      host_since      host_location ... \
0  2008-09-09  New York, New York, United States ...
1  2008-12-07  New York, New York, United States ...
2  2009-02-03  New York, New York, United States ...
3  2009-02-03  Brooklyn, New York, United States ...
4  2009-03-03  New York, New York, United States ...

      calculated_host_listings_count_entire_homes \
0      3
1      1
2      0
3      1
4      0

      calculated_host_listings_count_private_rooms \
0      0
```


1	0
2	2
3	0
4	1

	calculated_host_listings_count_shared_rooms	reviews_per_month	label_price	\
0	0	0.33	150.0	
1	0	4.86	75.0	
2	0	0.52	60.0	
3	0	0.02	275.0	
4	0	3.68	68.0	

	host_listings_count_na	host_total_listings_count_na	bathrooms_na	\
0	False	False	True	
1	False	False	True	
2	False	False	True	
3	False	False	True	
4	False	False	True	

	bedrooms_na	beds_na
0	True	False
1	False	False
2	False	False
3	False	False
4	False	False

[5 rows x 75 columns]

Replacing the missing values with mean values of the column Task: For every column listed in `to_impute_selected`, fill the missing values with the corresponding mean of all values in the column (do not create new columns).

```
[ ]: # YOUR CODE HERE

### Solution:
for colname in to_impute_selected:
    df[colname].fillna(np.mean(df[colname])), inplace=True)
```

Check your results below. The code displays the count of missing values for each of the selected columns.

```
[ ]: for colname in to_impute_selected:
    print("{} missing values count :{}".format(colname, np.sum(df[colname].
    →isnull(), axis = 0)))
```

```
host_listings_count missing values count :0
host_total_listings_count missing values count :0
```

```
bathrooms missing values count :38277
bedrooms missing values count :0
beds missing values count :0
```

Why did the `bathrooms` column retain missing values after our imputation?

Task: List the unique values of the `bathrooms` column.

```
[ ]: # YOUR CODE HERE
    ### Solution:
    df['bathrooms'].unique()
```

```
[ ]: array([nan])
```

The column did not contain a single value (except the NaN indicator) to begin with.

1.4 Part 3. Perform One-Hot Encoding

Machine learning algorithms operate on numerical inputs. Therefore, we have to transform text data into some form of numerical representation to prepare our data for the model training phase. Some features that contain text data are categorical. Others are not. For example, we removed all of the features that contained URLs. These features were not categorical, but rather contained what is called unstructured text. However, not all features that contain unstructured text should be removed, as they can contain useful information for our machine learning problem. Unstructured text data is usually handled by Natural Language Processing (NLP) techniques. You will learn more about NLP later in this course.

However, for features that contain categorical values, one-hot encoding is a common feature engineering technique that transforms them into binary representations.

We will first choose one feature column to one-hot encode: `host_response_time`. Let's inspect the unique values this feature can have.

```
[ ]: df['host_response_time'].unique()

[ ]: array(['within a day', 'a few days or more', 'within an hour', nan,
          'within a few hours'], dtype=object)
```

Note that each entry can contain one of five possible values.

Task: Since one of these values is NaN, replace every entry in the column `host_response_time` that contains a NaN value with the string 'unavailable'.

```
[ ]: # YOUR CODE HERE

    ### SOLUTION:

    df['host_response_time'].fillna('unavailable', inplace=True)
```

Let's inspect the `host_response_time` column to see the new values.

```
[ ]: df['host_response_time'].unique()
```

```
[ ]: array(['within a day', 'a few days or more', 'within an hour',
          'unavailable', 'within a few hours'], dtype=object)
```

Task: Use `pd.get_dummies()` to one-hot encode the `host_response_time` column. Save the result to DataFrame `df_host_response_time`.

```
[ ]: #df_host_response_time = # YOUR CODE HERE
      #df_host_response_time

### SOLUTION:
df_host_response_time = pd.get_dummies(df['host_response_time'],
    ↪ prefix='host_response_time_')
df_host_response_time
```

```
[ ]:      host_response_time__a few days or more \
0          0
1          1
2          0
3          0
4          0
...          ...
38272       0
38273       0
38274       0
38275       0
38276       0

      host_response_time__unavailable  host_response_time__within a day \
0          0          1
1          0          0
2          0          0
3          0          1
4          0          1
...          ...          ...
38272       0          0
38273       0          0
38274       0          0
38275       0          0
38276       0          0

      host_response_time__within a few hours \
0          0
1          0
2          0
```

3	0
4	0
...	...
38272	1
38273	1
38274	0
38275	0
38276	0

	host_response_time__within an hour
0	0
1	0
2	1
3	0
4	0
...	...
38272	0
38273	0
38274	1
38275	1
38276	1

[38277 rows x 5 columns]

Task: Since the `pd.get_dummies()` function returned a new DataFrame rather than making the changes to the original DataFrame `df`, add the new DataFrame `df_host_response_time` to DataFrame `df`, and delete the original `host_response_time` column from DataFrame `df`.

```
[ ]: # YOUR CODE HERE

### SOLUTION:

# Concatenate DataFrame df with the one-hot encoded DataFrame df_room_type
df = df.join(df_host_response_time)

# Remove the original 'Married' column from DataFrame df
df.drop(columns = 'host_response_time', inplace=True)
```

Let's inspect DataFrame `df` to see the changes that have been made.

```
[ ]: df.columns
```

```
[ ]: Index(['id', 'scrape_id', 'last_scraped', 'name', 'description',
          'neighborhood_overview', 'host_id', 'host_name', 'host_since',
          'host_location', 'host_about', 'host_response_rate',
          'host_acceptance_rate', 'host_is_superhost', 'host_neighbourhood',
          'host_listings_count', 'host_total_listings_count',
          'host_verifications', 'host_has_profile_pic', 'host_identity_verified',
```

```

'neighbourhood', 'neighbourhood_cleansed',
'neighbourhood_group_cleansed', 'latitude', 'longitude',
'property_type', 'room_type', 'accommodates', 'bathrooms',
'bathrooms_text', 'bedrooms', 'beds', 'amenities', 'price',
'minimum_nights', 'maximum_nights', 'minimum_minimum_nights',
'maximum_minimum_nights', 'minimum_maximum_nights',
'maximum_maximum_nights', 'minimum_nights_avg_ntm',
'maximum_nights_avg_ntm', 'calendar_updated', 'has_availability',
'availability_30', 'availability_60', 'availability_90',
'availability_365', 'calendar_last_scraped', 'number_of_reviews',
'number_of_reviews_ltm', 'number_of_reviews_l30d', 'first_review',
'last_review', 'review_scores_rating', 'review_scores_accuracy',
'review_scores_cleanliness', 'review_scores_checkin',
'review_scores_communication', 'review_scores_location',
'review_scores_value', 'license', 'instant_bookable',
'calculated_host_listings_count',
'calculated_host_listings_count_entire_homes',
'calculated_host_listings_count_private_rooms',
'calculated_host_listings_count_shared_rooms', 'reviews_per_month',
'label_price', 'host_listings_count_na', 'host_total_listings_count_na',
'bathrooms_na', 'bedrooms_na', 'beds_na',
'host_response_time__a few days or more',
'host_response_time__unavailable', 'host_response_time__within a day',
'host_response_time__within a few hours',
'host_response_time__within an hour'],
dtype='object')

```

One-hot encode additional features Task: Use the code cell below to find columns that contain string values (the 'object' data type) and inspect the *number* of unique values each column has.

```
[ ]: # YOUR CODE HERE
```

```

# Solution
to_encode = list(df.select_dtypes(include=['object']).columns)
df[to_encode].nunique()

```

```

[ ]: last_scraped          2
     name                 36870
     description          34133
     neighborhood_overview 18616
     host_name            9123
     host_since           4289
     host_location        1747
     host_about           14424
     host_response_rate    88

```

host_acceptance_rate	101
host_is_superhost	2
host_neighbourhood	484
host_verifications	526
host_has_profile_pic	2
host_identity_verified	2
neighbourhood	207
neighbourhood_cleansed	222
neighbourhood_group_cleansed	5
property_type	78
room_type	4
bathrooms_text	30
amenities	31740
has_availability	2
calendar_last_scraped	2
first_review	3171
last_review	2560
license	1
instant_bookable	2
dtype: int64	

Task: Based on your findings, identify features that you think should be transformed using one-hot encoding.

1. Use the code cell below to inspect the unique *values* that each of these features have.

```
[ ]: # YOUR CODE HERE

# SOLUTION (may vary, but the two features below are the most logical ones to
→ one-hot encode)
print(df['neighbourhood_group_cleansed'].unique())
print(df['room_type'].unique())
```

```
['Manhattan' 'Brooklyn' 'Queens' 'Staten Island' 'Bronx']
['Entire home/apt' 'Private room' 'Hotel room' 'Shared room']
```

2. List these features and explain why they would be suitable for one-hot encoding. Note your findings in the markdown cell below.

1.4.1 Solution:

Solutions may vary but the two features above are the most logical to one-hot encode. They each contain a few unique values which indicate that these features contain categorical values. Other text fields are not suitable for one-hot encoding.

Task: In the code cell below, one-hot encode one of the features you have identified and replace the original column in DataFrame `df` with the new one-hot encoded columns.

```
[ ]: # YOUR CODE HERE

# SOLUTION (may vary, but the two features below are the most logical ones to
→ one-hot encode)
df_room_type = pd.get_dummies(df['room_type'], prefix='room_type_')

# Concatenate DataFrame df with the one-hot encoded DataFrame df_room_type
df = df.join(df_room_type)

# Remove the original 'room_type' column from DataFrame df
df.drop(columns = 'room_type', inplace=True)
```

1.5 Part 4. Explore Your Data

You will now perform exploratory data analysis in preparation for selecting your features as part of feature engineering.

Identify Correlations We will focus on identifying which features in the data have the highest correlation with the label.

Let's first run the `corr()` method on DataFrame `df` and save the result to the variable `corr_matrix`. Let's round the resulting correlations to five decimal places:

```
[ ]: corr_matrix = round(df.corr(),5)
corr_matrix
```

```
[ ]:
```

	id	scrape_id	host_id	\
id	1.00000	-0.0	0.58617	
scrape_id	-0.00000	1.0	0.00000	
host_id	0.58617	0.0	1.00000	
host_listings_count	0.12986	-0.0	0.03189	
host_total_listings_count	0.12986	-0.0	0.03189	
latitude	0.01000	0.0	0.04148	
longitude	0.08708	-0.0	0.11620	
accommodates	0.03540	0.0	0.02723	
bathrooms	NaN	NaN	NaN	
bedrooms	0.04503	0.0	0.02202	
beds	0.03289	0.0	0.03689	
price	0.04256	-0.0	0.02907	
minimum_nights	-0.12067	0.0	-0.10640	
maximum_nights	-0.00696	0.0	-0.00385	
minimum_minimum_nights	-0.10234	0.0	-0.09188	
maximum_minimum_nights	-0.00041	-0.0	-0.04521	
minimum_maximum_nights	0.00747	-0.0	0.02572	
maximum_maximum_nights	0.01461	0.0	0.04267	

minimum_nights_avg_ntm	-0.00338	-0.0	-0.04707
maximum_nights_avg_ntm	0.01149	0.0	0.03438
calendar_updated	NaN	NaN	NaN
availability_30	0.25190	-0.0	0.26850
availability_60	0.32793	-0.0	0.32728
availability_90	0.34401	-0.0	0.33395
availability_365	0.28722	0.0	0.27332
number_of_reviews	-0.29164	0.0	-0.12215
number_of_reviews_ltm	0.07737	0.0	0.11469
number_of_reviews_l30d	0.15257	-0.0	0.15333
review_scores_rating	0.01187	0.0	-0.04397
review_scores_accuracy	-0.08867	0.0	-0.15428
review_scores_cleanliness	0.00424	0.0	-0.05183
review_scores_checkin	-0.09156	0.0	-0.14890
review_scores_communication	-0.11950	0.0	-0.17420
review_scores_location	0.00322	0.0	-0.07864
review_scores_value	-0.07080	0.0	-0.13340
calculated_host_listings_count	0.23667	-0.0	0.15754
calculated_host_listings_count_entire_homes	0.13713	0.0	0.02524
calculated_host_listings_count_private_rooms	0.21188	-0.0	0.19320
calculated_host_listings_count_shared_rooms	0.04671	-0.0	0.07831
reviews_per_month	0.23169	0.0	0.20844
label_price	0.07907	-0.0	0.04053
host_listings_count_na	-0.00830	-0.0	-0.00371
host_total_listings_count_na	-0.00830	-0.0	-0.00371
bathrooms_na	NaN	NaN	NaN
bedrooms_na	0.03343	0.0	0.03354
beds_na	0.13640	-0.0	0.09218
host_response_time__a few days or more	0.01215	0.0	0.04055
host_response_time__unavailable	-0.35410	-0.0	-0.24094
host_response_time__within a day	-0.01164	-0.0	-0.05562
host_response_time__within a few hours	0.12780	-0.0	0.01844
host_response_time__within an hour	0.29187	-0.0	0.26491
room_type__Entire home/apt	-0.04284	-0.0	-0.12862
room_type__Hotel room	0.01698	0.0	0.07086
room_type__Private room	0.03813	0.0	0.10957
room_type__Shared room	0.00958	0.0	0.03676

	host_listings_count \
id	0.12986
scrape_id	-0.00000
host_id	0.03189
host_listings_count	1.00000
host_total_listings_count	1.00000
latitude	0.03475
longitude	-0.08843
accommodates	-0.02621

bathrooms	NaN
bedrooms	-0.01710
beds	-0.03151
price	0.07492
minimum_nights	0.19739
maximum_nights	-0.00080
minimum_minimum_nights	0.26125
maximum_minimum_nights	0.65300
minimum_maximum_nights	-0.00349
maximum_maximum_nights	-0.00529
minimum_nights_avg_ntm	0.65239
maximum_nights_avg_ntm	-0.00451
calendar_updated	NaN
availability_30	0.07148
availability_60	0.06218
availability_90	0.06279
availability_365	0.14287
number_of_reviews	-0.06617
number_of_reviews_ltm	-0.04448
number_of_reviews_l30d	-0.04962
review_scores_rating	-0.00742
review_scores_accuracy	-0.02365
review_scores_cleanliness	-0.00694
review_scores_checkin	-0.01701
review_scores_communication	-0.05032
review_scores_location	0.00638
review_scores_value	-0.07391
calculated_host_listings_count	0.42944
calculated_host_listings_count_entire_homes	0.54188
calculated_host_listings_count_private_rooms	0.14915
calculated_host_listings_count_shared_rooms	-0.01595
reviews_per_month	-0.02096
label_price	0.13104
host_listings_count_na	-0.00000
host_total_listings_count_na	-0.00000
bathrooms_na	NaN
bedrooms_na	0.01297
beds_na	-0.01032
host_response_time__a few days or more	-0.03124
host_response_time__unavailable	-0.11686
host_response_time__within a day	-0.03119
host_response_time__within a few hours	-0.01468
host_response_time__within an hour	0.17132
room_type__Entire home/apt	0.01040
room_type__Hotel room	-0.00877
room_type__Private room	-0.00468
room_type__Shared room	-0.01825

	host_total_listings_count \
id	0.12986
scrape_id	-0.00000
host_id	0.03189
host_listings_count	1.00000
host_total_listings_count	1.00000
latitude	0.03475
longitude	-0.08843
accommodates	-0.02621
bathrooms	NaN
bedrooms	-0.01710
beds	-0.03151
price	0.07492
minimum_nights	0.19739
maximum_nights	-0.00080
minimum_minimum_nights	0.26125
maximum_minimum_nights	0.65300
minimum_maximum_nights	-0.00349
maximum_maximum_nights	-0.00529
minimum_nights_avg_ntm	0.65239
maximum_nights_avg_ntm	-0.00451
calendar_updated	NaN
availability_30	0.07148
availability_60	0.06218
availability_90	0.06279
availability_365	0.14287
number_of_reviews	-0.06617
number_of_reviews_ltm	-0.04448
number_of_reviews_l30d	-0.04962
review_scores_rating	-0.00742
review_scores_accuracy	-0.02365
review_scores_cleanliness	-0.00694
review_scores_checkin	-0.01701
review_scores_communication	-0.05032
review_scores_location	0.00638
review_scores_value	-0.07391
calculated_host_listings_count	0.42944
calculated_host_listings_count_entire_homes	0.54188
calculated_host_listings_count_private_rooms	0.14915
calculated_host_listings_count_shared_rooms	-0.01595
reviews_per_month	-0.02096
label_price	0.13104
host_listings_count_na	-0.00000
host_total_listings_count_na	-0.00000
bathrooms_na	NaN
bedrooms_na	0.01297

beds_na	-0.01032
host_response_time__a few days or more	-0.03124
host_response_time__unavailable	-0.11686
host_response_time__within a day	-0.03119
host_response_time__within a few hours	-0.01468
host_response_time__within an hour	0.17132
room_type__Entire home/apt	0.01040
room_type__Hotel room	-0.00877
room_type__Private room	-0.00468
room_type__Shared room	-0.01825

	latitude	longitude \
id	0.01000	0.08708
scrape_id	0.00000	-0.00000
host_id	0.04148	0.11620
host_listings_count	0.03475	-0.08843
host_total_listings_count	0.03475	-0.08843
latitude	1.00000	0.05718
longitude	0.05718	1.00000
accommodates	-0.04745	0.00374
bathrooms	NaN	NaN
bedrooms	-0.07150	0.00752
beds	-0.05388	0.03136
price	0.02734	-0.11484
minimum_nights	0.03422	-0.08550
maximum_nights	0.00561	-0.00296
minimum_minimum_nights	0.03317	-0.08397
maximum_minimum_nights	0.04352	-0.09520
minimum_maximum_nights	0.01735	-0.00780
maximum_maximum_nights	0.01598	-0.01993
minimum_nights_avg_ntm	0.04379	-0.09507
maximum_nights_avg_ntm	0.01828	-0.01401
calendar_updated	NaN	NaN
availability_30	0.00261	0.13025
availability_60	0.00026	0.15062
availability_90	-0.00157	0.14953
availability_365	0.01383	0.09596
number_of_reviews	-0.04801	0.06759
number_of_reviews_ltm	-0.04884	0.06458
number_of_reviews_l30d	-0.04339	0.07309
review_scores_rating	-0.03767	0.00523
review_scores_accuracy	-0.04076	-0.01136
review_scores_cleanliness	-0.03469	0.00772
review_scores_checkin	-0.04612	-0.00525
review_scores_communication	-0.04250	-0.01358
review_scores_location	0.01355	-0.13822
review_scores_value	-0.04887	0.00052

calculated_host_listings_count	0.07954	-0.06543
calculated_host_listings_count_entire_homes	0.07065	-0.12713
calculated_host_listings_count_private_rooms	0.05096	0.01401
calculated_host_listings_count_shared_rooms	0.00762	0.02066
reviews_per_month	-0.03667	0.07121
label_price	0.04330	-0.20695
host_listings_count_na	0.00199	-0.01261
host_total_listings_count_na	0.00199	-0.01261
bathrooms_na	NaN	NaN
bedrooms_na	0.05533	-0.10992
beds_na	0.02258	0.00221
host_response_time__a few days or more	0.02052	-0.01400
host_response_time__unavailable	0.01134	-0.07471
host_response_time__within a day	0.01410	-0.03805
host_response_time__within a few hours	-0.00499	0.03534
host_response_time__within an hour	-0.02598	0.08358
room_type__Entire home/apt	-0.02656	-0.14909
room_type__Hotel room	0.02825	-0.04860
room_type__Private room	0.01830	0.15128
room_type__Shared room	0.01707	0.02280

	accommodates	bathrooms	\
id	0.03540	NaN	
scrape_id	0.00000	NaN	
host_id	0.02723	NaN	
host_listings_count	-0.02621	NaN	
host_total_listings_count	-0.02621	NaN	
latitude	-0.04745	NaN	
longitude	0.00374	NaN	
accommodates	1.00000	NaN	
bathrooms	NaN	NaN	
bedrooms	0.70586	NaN	
beds	0.73665	NaN	
price	0.30803	NaN	
minimum_nights	-0.08474	NaN	
maximum_nights	-0.00494	NaN	
minimum_minimum_nights	-0.07485	NaN	
maximum_minimum_nights	-0.05134	NaN	
minimum_maximum_nights	-0.00249	NaN	
maximum_maximum_nights	-0.00931	NaN	
minimum_nights_avg_ntm	-0.05266	NaN	
maximum_nights_avg_ntm	-0.00558	NaN	
calendar_updated	NaN	NaN	
availability_30	0.04429	NaN	
availability_60	0.07983	NaN	
availability_90	0.09096	NaN	
availability_365	0.10293	NaN	

number_of_reviews	0.07255	NaN
number_of_reviews_ltm	0.08118	NaN
number_of_reviews_l30d	0.08552	NaN
review_scores_rating	0.03097	NaN
review_scores_accuracy	-0.00422	NaN
review_scores_cleanliness	0.03702	NaN
review_scores_checkin	-0.00125	NaN
review_scores_communication	-0.00067	NaN
review_scores_location	-0.01220	NaN
review_scores_value	-0.00778	NaN
calculated_host_listings_count	-0.11818	NaN
calculated_host_listings_count_entire_homes	-0.01929	NaN
calculated_host_listings_count_private_rooms	-0.14499	NaN
calculated_host_listings_count_shared_rooms	-0.05161	NaN
reviews_per_month	0.06850	NaN
label_price	0.50062	NaN
host_listings_count_na	0.00519	NaN
host_total_listings_count_na	0.00519	NaN
bathrooms_na	NaN	NaN
bedrooms_na	-0.05957	NaN
beds_na	-0.06916	NaN
host_response_time__a few days or more	0.01101	NaN
host_response_time__unavailable	-0.11168	NaN
host_response_time__within a day	0.01642	NaN
host_response_time__within a few hours	-0.00382	NaN
host_response_time__within an hour	0.11060	NaN
room_type__Entire home/apt	0.45742	NaN
room_type__Hotel room	-0.01671	NaN
room_type__Private room	-0.44105	NaN
room_type__Shared room	-0.06358	NaN

	bedrooms	...	beds_na	\
id	0.04503	...	0.13640	
scrape_id	0.00000	...	-0.00000	
host_id	0.02202	...	0.09218	
host_listings_count	-0.01710	...	-0.01032	
host_total_listings_count	-0.01710	...	-0.01032	
latitude	-0.07150	...	0.02258	
longitude	0.00752	...	0.00221	
accommodates	0.70586	...	-0.06916	
bathrooms	NaN	...	NaN	
bedrooms	1.00000	...	-0.04571	
beds	0.72914	...	0.00000	
price	0.25383	...	-0.01596	
minimum_nights	-0.02749	...	-0.01830	
maximum_nights	0.00002	...	-0.00135	
minimum_minimum_nights	-0.02546	...	-0.01823	

maximum_minimum_nights	-0.01708	...	-0.02851
minimum_maximum_nights	-0.01161	...	-0.00673
maximum_maximum_nights	-0.01705	...	-0.00781
minimum_nights_avg_ntm	-0.01782	...	-0.02848
maximum_nights_avg_ntm	-0.01465	...	-0.00831
calendar_updated	NaN	...	NaN
availability_30	0.01816	...	0.09611
availability_60	0.04432	...	0.09098
availability_90	0.05567	...	0.09143
availability_365	0.08280	...	0.08961
number_of_reviews	0.00408	...	-0.05311
number_of_reviews_ltm	0.02836	...	-0.03291
number_of_reviews_l30d	0.03271	...	-0.01860
review_scores_rating	0.01686	...	-0.01925
review_scores_accuracy	-0.00323	...	-0.04077
review_scores_cleanliness	0.03206	...	-0.03027
review_scores_checkin	0.00638	...	-0.04050
review_scores_communication	-0.00019	...	-0.03904
review_scores_location	-0.01053	...	-0.02043
review_scores_value	0.00074	...	-0.03429
calculated_host_listings_count	-0.05754	...	0.12938
calculated_host_listings_count_entire_homes	-0.00212	...	0.01163
calculated_host_listings_count_private_rooms	-0.07591	...	0.16732
calculated_host_listings_count_shared_rooms	-0.04902	...	0.01101
reviews_per_month	0.03030	...	-0.00329
label_price	0.41996	...	-0.03461
host_listings_count_na	-0.00089	...	-0.00772
host_total_listings_count_na	-0.00089	...	-0.00772
bathrooms_na	NaN	...	NaN
bedrooms_na	-0.00000	...	0.04985
beds_na	-0.04571	...	1.00000
host_response_time__a few days or more	0.01969	...	0.04385
host_response_time__unavailable	-0.09343	...	-0.02452
host_response_time__within a day	0.03512	...	0.02050
host_response_time__within a few hours	0.01114	...	0.00105
host_response_time__within an hour	0.06432	...	-0.00536
room_type__Entire home/apt	0.35604	...	-0.06572
room_type__Hotel room	-0.02448	...	0.03615
room_type__Private room	-0.33917	...	0.05451
room_type__Shared room	-0.05944	...	0.02490

more \

id

0.01215

scrape_id

0.00000

host_response_time__a few days or

host_id
0.04055
host_listings_count
-0.03124
host_total_listings_count
-0.03124
latitude
0.02052
longitude
-0.01400
accommodates
0.01101
bathrooms
NaN
bedrooms
0.01969
beds
0.02056
price
0.02432
minimum_nights
0.03087
maximum_nights
-0.00108
minimum_minimum_nights
0.02434
maximum_minimum_nights
-0.00457
minimum_maximum_nights
-0.00543
maximum_maximum_nights
-0.00845
minimum_nights_avg_ntm
-0.00364
maximum_nights_avg_ntm
-0.00717
calendar_updated
NaN
availability_30
0.20254
availability_60
0.18352
availability_90
0.17710
availability_365
0.12545
number_of_reviews

-0.03115
number_of_reviews_ltm
-0.06060
number_of_reviews_l30d
-0.07216
review_scores_rating
-0.06101
review_scores_accuracy
-0.07606
review_scores_cleanliness
-0.06482
review_scores_checkin
-0.08196
review_scores_communication
-0.08031
review_scores_location
-0.04102
review_scores_value
-0.06118
calculated_host_listings_count
-0.05406
calculated_host_listings_count_entire_homes
-0.04190
calculated_host_listings_count_private_rooms
-0.03991
calculated_host_listings_count_shared_rooms
0.02082
reviews_per_month
-0.04892
label_price
0.00792
host_listings_count_na
-0.00620
host_total_listings_count_na
-0.00620
bathrooms_na
NaN
bedrooms_na
-0.00898
beds_na
0.04385
host_response_time__a few days or more
1.00000
host_response_time__unavailable
-0.18787
host_response_time__within a day
-0.06088

host_response_time__within a few hours
 -0.08364
 host_response_time__within an hour
 -0.13339
 room_type__Entire home/apt
 -0.00872
 room_type__Hotel room
 -0.01191
 room_type__Private room
 0.00571
 room_type__Shared room
 0.01973

	host_response_time__unavailable \
id	-0.35410
scrape_id	-0.00000
host_id	-0.24094
host_listings_count	-0.11686
host_total_listings_count	-0.11686
latitude	0.01134
longitude	-0.07471
accommodates	-0.11168
bathrooms	NaN
bedrooms	-0.09343
beds	-0.10810
price	-0.05266
minimum_nights	0.18254
maximum_nights	0.00577
minimum_minimum_nights	0.15024
maximum_minimum_nights	0.00076
minimum_maximum_nights	-0.00942
maximum_maximum_nights	-0.02371
minimum_nights_avg_ntm	0.00547
maximum_nights_avg_ntm	-0.01380
calendar_updated	NaN
availability_30	-0.29428
availability_60	-0.43295
availability_90	-0.47929
availability_365	-0.47520
number_of_reviews	-0.16121
number_of_reviews_ltm	-0.22794
number_of_reviews_l30d	-0.24822
review_scores_rating	-0.09901
review_scores_accuracy	0.04080
review_scores_cleanliness	-0.06196
review_scores_checkin	0.02230
review_scores_communication	0.05199

review_scores_location	0.01118
review_scores_value	0.04111
calculated_host_listings_count	-0.08352
calculated_host_listings_count_entire_homes	-0.14256
calculated_host_listings_count_private_rooms	0.00213
calculated_host_listings_count_shared_rooms	-0.01928
reviews_per_month	-0.20592
label_price	-0.10279
host_listings_count_na	0.03302
host_total_listings_count_na	0.03302
bathrooms_na	NaN
bedrooms_na	-0.02418
beds_na	-0.02452
host_response_time__a few days or more	-0.18787
host_response_time__unavailable	1.00000
host_response_time__within a day	-0.26424
host_response_time__within a few hours	-0.36305
host_response_time__within an hour	-0.57898
room_type__Entire home/apt	-0.04946
room_type__Hotel room	-0.03010
room_type__Private room	0.05008
room_type__Shared room	0.01648

host_response_time__within a day

\	
id	-0.01164
scrape_id	-0.00000
host_id	-0.05562
host_listings_count	-0.03119
host_total_listings_count	-0.03119
latitude	0.01410
longitude	-0.03805
accommodates	0.01642
bathrooms	NaN
bedrooms	0.03512
beds	0.01886
price	-0.00026
minimum_nights	-0.00695
maximum_nights	-0.00153
minimum_minimum_nights	-0.01002
maximum_minimum_nights	-0.02714
minimum_maximum_nights	0.02956
maximum_maximum_nights	0.02398
minimum_nights_avg_ntm	-0.02691
maximum_nights_avg_ntm	0.02215
calendar_updated	NaN
availability_30	0.04232

availability_60	0.05946
availability_90	0.08130
availability_365	0.10797
number_of_reviews	0.00818
number_of_reviews_ltm	-0.03950
number_of_reviews_l30d	-0.05445
review_scores_rating	0.02862
review_scores_accuracy	0.00761
review_scores_cleanliness	0.01355
review_scores_checkin	0.01290
review_scores_communication	-0.00556
review_scores_location	0.00999
review_scores_value	-0.00564
calculated_host_listings_count	-0.01243
calculated_host_listings_count_entire_homes	0.04999
calculated_host_listings_count_private_rooms	-0.05728
calculated_host_listings_count_shared_rooms	-0.01131
reviews_per_month	-0.04801
label_price	0.01335
host_listings_count_na	-0.00873
host_total_listings_count_na	-0.00873
bathrooms_na	NaN
bedrooms_na	0.02494
beds_na	0.02050
host_response_time__a few days or more	-0.06088
host_response_time__unavailable	-0.26424
host_response_time__within a day	1.00000
host_response_time__within a few hours	-0.11764
host_response_time__within an hour	-0.18761
room_type__Entire home/apt	0.06668
room_type__Hotel room	0.00451
room_type__Private room	-0.06601
room_type__Shared room	-0.00648

host_response_time__within a few

hours \

id	
0.12780	
scrape_id	
-0.00000	
host_id	
0.01844	
host_listings_count	
-0.01468	
host_total_listings_count	
-0.01468	
latitude	

-0.00499
longitude
0.03534
accommodates
-0.00382
bathrooms
NaN
bedrooms
0.01114
beds
0.00242
price
-0.01433
minimum_nights
0.00592
maximum_nights
-0.00210
minimum_minimum_nights
-0.00678
maximum_minimum_nights
-0.02551
minimum_maximum_nights
-0.01049
maximum_maximum_nights
-0.01634
minimum_nights_avg_ntm
-0.02692
maximum_nights_avg_ntm
-0.01386
calendar_updated
NaN
availability_30
0.10312
availability_60
0.13745
availability_90
0.14265
availability_365
0.17218
number_of_reviews
-0.00846
number_of_reviews_ltm
-0.02346
number_of_reviews_l30d
-0.02925
review_scores_rating
0.02229

review_scores_accuracy
-0.04651
review_scores_cleanliness
-0.01300
review_scores_checkin
-0.01974
review_scores_communication
-0.04243
review_scores_location
-0.01360
review_scores_value
-0.05498
calculated_host_listings_count
0.09949
calculated_host_listings_count_entire_homes
0.01930
calculated_host_listings_count_private_rooms
0.11927
calculated_host_listings_count_shared_rooms
0.01389
reviews_per_month
-0.02663
label_price
-0.02111
host_listings_count_na
-0.01199
host_total_listings_count_na
-0.01199
bathrooms_na
NaN
bedrooms_na
-0.00819
beds_na
0.00105
host_response_time__a few days or more
-0.08364
host_response_time__unavailable
-0.36305
host_response_time__within a day
-0.11764
host_response_time__within a few hours
1.00000
host_response_time__within an hour
-0.25777
room_type__Entire home/apt
0.00195
room_type__Hotel room

-0.01658
room_type__Private room
0.00196
room_type__Shared room
-0.00597

host_response_time__within an hour

\	
id	0.29187
scrape_id	-0.00000
host_id	0.26491
host_listings_count	0.17132
host_total_listings_count	0.17132
latitude	-0.02598
longitude	0.08358
accommodates	0.11060
bathrooms	NaN
bedrooms	0.06432
beds	0.09628
price	0.05805
minimum_nights	-0.21377
maximum_nights	-0.00334
minimum_minimum_nights	-0.16408
maximum_minimum_nights	0.03672
minimum_maximum_nights	0.00315
maximum_maximum_nights	0.02789
minimum_nights_avg_ntm	0.03209
maximum_nights_avg_ntm	0.01568
calendar_updated	NaN
availability_30	0.12962
availability_60	0.25344
availability_90	0.29008
availability_365	0.26996
number_of_reviews	0.19174
number_of_reviews_ltm	0.31743
number_of_reviews_l30d	0.35797
review_scores_rating	0.09629
review_scores_accuracy	0.01555
review_scores_cleanliness	0.09180
review_scores_checkin	0.01498
review_scores_communication	0.01028
review_scores_location	0.00806
review_scores_value	0.02334
calculated_host_listings_count	0.04675
calculated_host_listings_count_entire_homes	0.13010
calculated_host_listings_count_private_rooms	-0.04169
calculated_host_listings_count_shared_rooms	0.00810

reviews_per_month	0.28524
label_price	0.11721
host_listings_count_na	-0.01912
host_total_listings_count_na	-0.01912
bathrooms_na	NaN
bedrooms_na	0.02186
beds_na	-0.00536
host_response_time__a few days or more	-0.13339
host_response_time__unavailable	-0.57898
host_response_time__within a day	-0.18761
host_response_time__within a few hours	-0.25777
host_response_time__within an hour	1.00000
room_type__Entire home/apt	0.01693
room_type__Hotel room	0.04812
room_type__Private room	-0.01967
room_type__Shared room	-0.01831

	room_type__Entire home/apt \
id	-0.04284
scrape_id	-0.00000
host_id	-0.12862
host_listings_count	0.01040
host_total_listings_count	0.01040
latitude	-0.02656
longitude	-0.14909
accommodates	0.45742
bathrooms	NaN
bedrooms	0.35604
beds	0.32487
price	0.17365
minimum_nights	0.00925
maximum_nights	0.00478
minimum_minimum_nights	0.02079
maximum_minimum_nights	0.07891
minimum_maximum_nights	-0.02184
maximum_maximum_nights	-0.03952
minimum_nights_avg_ntm	0.07834
maximum_nights_avg_ntm	-0.03164
calendar_updated	NaN
availability_30	-0.10800
availability_60	-0.08439
availability_90	-0.06442
availability_365	-0.00816
number_of_reviews	0.02319
number_of_reviews_ltm	0.02510
number_of_reviews_l30d	0.03656
review_scores_rating	0.08109

review_scores_accuracy	0.09148
review_scores_cleanliness	0.10695
review_scores_checkin	0.07370
review_scores_communication	0.08425
review_scores_location	0.09444
review_scores_value	0.04539
calculated_host_listings_count	-0.04794
calculated_host_listings_count_entire_homes	0.16276
calculated_host_listings_count_private_rooms	-0.19529
calculated_host_listings_count_shared_rooms	-0.11059
reviews_per_month	-0.00268
label_price	0.33529
host_listings_count_na	0.01561
host_total_listings_count_na	0.01561
bathrooms_na	NaN
bedrooms_na	0.20509
beds_na	-0.06572
host_response_time__a few days or more	-0.00872
host_response_time__unavailable	-0.04946
host_response_time__within a day	0.06668
host_response_time__within a few hours	0.00195
host_response_time__within an hour	0.01693
room_type__Entire home/apt	1.00000
room_type__Hotel room	-0.07933
room_type__Private room	-0.95966
room_type__Shared room	-0.13155

	room_type__Hotel room \
id	0.01698
scrape_id	0.00000
host_id	0.07086
host_listings_count	-0.00877
host_total_listings_count	-0.00877
latitude	0.02825
longitude	-0.04860
accommodates	-0.01671
bathrooms	NaN
bedrooms	-0.02448
beds	-0.01256
price	0.05119
minimum_nights	-0.03447
maximum_nights	-0.00039
minimum_minimum_nights	-0.02844
maximum_minimum_nights	-0.01886
minimum_maximum_nights	0.14009
maximum_maximum_nights	0.11571
minimum_nights_avg_ntm	-0.01984

maximum_nights_avg_ntm	0.15595
calendar_updated	NaN
availability_30	0.04272
availability_60	0.03851
availability_90	0.03578
availability_365	0.05067
number_of_reviews	0.03582
number_of_reviews_ltm	0.08765
number_of_reviews_l30d	0.00086
review_scores_rating	-0.01071
review_scores_accuracy	-0.03556
review_scores_cleanliness	0.00819
review_scores_checkin	-0.02068
review_scores_communication	-0.02970
review_scores_location	0.01197
review_scores_value	-0.03393
calculated_host_listings_count	-0.00784
calculated_host_listings_count_entire_homes	-0.00853
calculated_host_listings_count_private_rooms	-0.01535
calculated_host_listings_count_shared_rooms	-0.00835
reviews_per_month	0.03322
label_price	0.10587
host_listings_count_na	-0.00221
host_total_listings_count_na	-0.00221
bathrooms_na	NaN
bedrooms_na	0.03037
beds_na	0.03615
host_response_time__a few days or more	-0.01191
host_response_time__unavailable	-0.03010
host_response_time__within a day	0.00451
host_response_time__within a few hours	-0.01658
host_response_time__within an hour	0.04812
room_type__Entire home/apt	-0.07933
room_type__Hotel room	1.00000
room_type__Private room	-0.06674
room_type__Shared room	-0.00915

	room_type__Private room \
id	0.03813
scrape_id	0.00000
host_id	0.10957
host_listings_count	-0.00468
host_total_listings_count	-0.00468
latitude	0.01830
longitude	0.15128
accommodates	-0.44105
bathrooms	NaN

bedrooms	-0.33917
beds	-0.32660
price	-0.18024
minimum_nights	-0.00313
maximum_nights	-0.00458
minimum_minimum_nights	-0.01574
maximum_minimum_nights	-0.07349
minimum_maximum_nights	0.00279
maximum_maximum_nights	0.02444
minimum_nights_avg_ntm	-0.07289
maximum_nights_avg_ntm	0.01063
calendar_updated	NaN
availability_30	0.08909
availability_60	0.06938
availability_90	0.05103
availability_365	-0.00435
number_of_reviews	-0.02639
number_of_reviews_ltm	-0.03482
number_of_reviews_l30d	-0.03389
review_scores_rating	-0.07572
review_scores_accuracy	-0.08241
review_scores_cleanliness	-0.10530
review_scores_checkin	-0.06553
review_scores_communication	-0.07540
review_scores_location	-0.09296
review_scores_value	-0.03770
calculated_host_listings_count	0.05666
calculated_host_listings_count_entire_homes	-0.15528
calculated_host_listings_count_private_rooms	0.20438
calculated_host_listings_count_shared_rooms	-0.04520
reviews_per_month	-0.00053
label_price	-0.34108
host_listings_count_na	-0.01444
host_total_listings_count_na	-0.01444
bathrooms_na	NaN
bedrooms_na	-0.20010
beds_na	0.05451
host_response_time__a few days or more	0.00571
host_response_time__unavailable	0.05008
host_response_time__within a day	-0.06601
host_response_time__within a few hours	0.00196
host_response_time__within an hour	-0.01967
room_type__Entire home/apt	-0.95966
room_type__Hotel room	-0.06674
room_type__Private room	1.00000
room_type__Shared room	-0.11067

	room_type__Shared room
id	0.00958
scrape_id	0.00000
host_id	0.03676
host_listings_count	-0.01825
host_total_listings_count	-0.01825
latitude	0.01707
longitude	0.02280
accommodates	-0.06358
bathrooms	NaN
bedrooms	-0.05944
beds	0.01000
price	-0.00669
minimum_nights	-0.00423
maximum_nights	-0.00064
minimum_minimum_nights	-0.00443
maximum_minimum_nights	-0.01233
minimum_maximum_nights	-0.00322
maximum_maximum_nights	-0.00500
minimum_nights_avg_ntm	-0.01192
maximum_nights_avg_ntm	-0.00425
calendar_updated	NaN
availability_30	0.05305
availability_60	0.03931
availability_90	0.03405
availability_365	0.02056
number_of_reviews	-0.00903
number_of_reviews_ltm	-0.01391
number_of_reviews_l30d	-0.01197
review_scores_rating	-0.01767
review_scores_accuracy	-0.01757
review_scores_cleanliness	-0.01476
review_scores_checkin	-0.02305
review_scores_communication	-0.02031
review_scores_location	-0.01618
review_scores_value	-0.01173
calculated_host_listings_count	-0.03027
calculated_host_listings_count_entire_homes	-0.02785
calculated_host_listings_count_private_rooms	-0.02503
calculated_host_listings_count_shared_rooms	0.64509
reviews_per_month	-0.00766
label_price	-0.04563
host_listings_count_na	-0.00367
host_total_listings_count_na	-0.00367
bathrooms_na	NaN
bedrooms_na	-0.04193
beds_na	0.02490

host_response_time__a few days or more	0.01973
host_response_time__unavailable	0.01648
host_response_time__within a day	-0.00648
host_response_time__within a few hours	-0.00597
host_response_time__within an hour	-0.01831
room_type__Entire home/apt	-0.13155
room_type__Hotel room	-0.00915
room_type__Private room	-0.11067
room_type__Shared room	1.00000

[55 rows x 55 columns]

The result is a computed *correlation matrix*. The values on the diagonal are all equal to 1 because they represent the correlations between each column with itself. The matrix is symmetrical with respect to the diagonal.

We only need to observe correlations of all features with the column `label_price` (as opposed to every possible pairwise correlation). So let's query the `label_price` column of this matrix:

Task: Extract the `label_price` column of the correlation matrix and save the results to the variable `corrs`.

```
[ ]: # corrs = # YOUR CODE HERE
# corrs

### Solution
corrs = corr_matrix['label_price']
corrs
```

```
[ ]: id          0.07907
scrape_id       -0.00000
host_id         0.04053
host_listings_count 0.13104
host_total_listings_count 0.13104
latitude        0.04330
longitude       -0.20695
accommodates    0.50062
bathrooms       NaN
bedrooms        0.41996
beds            0.37370
price           0.71112
minimum_nights  -0.07589
maximum_nights  -0.00097
minimum_minimum_nights -0.03804
maximum_minimum_nights 0.06554
minimum_maximum_nights 0.06582
maximum_maximum_nights 0.11169
minimum_nights_avg_ntm 0.06388
```

maximum_nights_avg_ntm	0.08210
calendar_updated	NaN
availability_30	0.14569
availability_60	0.14701
availability_90	0.14391
availability_365	0.12356
number_of_reviews	-0.04197
number_of_reviews_ltm	0.02757
number_of_reviews_l30d	0.02159
review_scores_rating	0.04320
review_scores_accuracy	0.00536
review_scores_cleanliness	0.08254
review_scores_checkin	-0.00367
review_scores_communication	0.00012
review_scores_location	0.09724
review_scores_value	-0.00482
calculated_host_listings_count	-0.01582
calculated_host_listings_count_entire_homes	0.09509
calculated_host_listings_count_private_rooms	-0.09978
calculated_host_listings_count_shared_rooms	-0.04334
reviews_per_month	0.03114
label_price	1.00000
host_listings_count_na	0.04450
host_total_listings_count_na	0.04450
bathrooms_na	NaN
bedrooms_na	0.02381
beds_na	-0.03461
host_response_time__a few days or more	0.00792
host_response_time__unavailable	-0.10279
host_response_time__within a day	0.01335
host_response_time__within a few hours	-0.02111
host_response_time__within an hour	0.11721
room_type__Entire home/apt	0.33529
room_type__Hotel room	0.10587
room_type__Private room	-0.34108
room_type__Shared room	-0.04563

Name: label_price, dtype: float64

Task: Sort the values of the series we just obtained in the descending order and save the results to the variable `corrs_sorted`.

```
[ ]: #corrs_sorted = # YOUR CODE HERE
      #corrs_sorted

corrs_sorted = corrs.sort_values(ascending=False)
corrs_sorted
```

[]: label_price	1.00000
price	0.71112
accommodates	0.50062
bedrooms	0.41996
beds	0.37370
room_type__Entire home/apt	0.33529
availability_60	0.14701
availability_30	0.14569
availability_90	0.14391
host_total_listings_count	0.13104
host_listings_count	0.13104
availability_365	0.12356
host_response_time__within an hour	0.11721
maximum_maximum_nights	0.11169
room_type__Hotel room	0.10587
review_scores_location	0.09724
calculated_host_listings_count_entire_homes	0.09509
review_scores_cleanliness	0.08254
maximum_nights_avg_ntm	0.08210
id	0.07907
minimum_maximum_nights	0.06582
maximum_minimum_nights	0.06554
minimum_nights_avg_ntm	0.06388
host_listings_count_na	0.04450
host_total_listings_count_na	0.04450
latitude	0.04330
review_scores_rating	0.04320
host_id	0.04053
reviews_per_month	0.03114
number_of_reviews_ltm	0.02757
bedrooms_na	0.02381
number_of_reviews_l30d	0.02159
host_response_time__within a day	0.01335
host_response_time__a few days or more	0.00792
review_scores_accuracy	0.00536
review_scores_communication	0.00012
scrape_id	-0.00000
maximum_nights	-0.00097
review_scores_checkin	-0.00367
review_scores_value	-0.00482
calculated_host_listings_count	-0.01582
host_response_time__within a few hours	-0.02111
beds_na	-0.03461
minimum_minimum_nights	-0.03804
number_of_reviews	-0.04197
calculated_host_listings_count_shared_rooms	-0.04334
room_type__Shared room	-0.04563

```

minimum_nights                    -0.07589
calculated_host_listings_count_private_rooms -0.09978
host_response_time__unavailable   -0.10279
longitude                        -0.20695
room_type__Private room          -0.34108
bathrooms                       NaN
calendar_updated                 NaN
bathrooms_na                     NaN
Name: label_price, dtype: float64

```

Task: Use Pandas indexing to extract the column names for the top two correlation values and save the results to the Python list `top_two_corr`. Add the feature names to the list in the order in which they appear in the output above.

Note: Do not count the correlation of `label` column with itself, nor the `price` column -- which is the `label` column prior to outlier removal.

```

[ ]: #top_two_corr = # YOUR CODE HERE
    #top_two_corr

# SOLUTION:
top_two_corr = list(corrs_sorted[2:4].index)
top_two_corr

```

```

[ ]: ['accommodates', 'bedrooms']

```

Bivariate Plotting: Produce Plots for the Label and Its Top Correlates Let us visualize our data.

We will use the `pairplot()` function in `seaborn` to plot the relationships between the two features and the label.

Task: Create a DataFrame `df_corrs` that contains only three columns from DataFrame `df`: the label, and the two columns which correlate with it the most.

```

[ ]: #df_corrs = # YOUR CODE HERE
    #df_corrs

### Solution (solutions may vary)
df_corrs = df[top_two_corr].copy()
df_corrs['label_price'] = df['label_price']
df_corrs

```

```

[ ]:      accommodates  bedrooms  label_price
0              1  1.323567      150.0
1              3  1.000000       75.0
2              2  1.000000       60.0
3              4  2.000000      275.0

```

4		2	1.000000	68.0
...	...	...	...	
38272		2	1.000000	79.0
38273		2	1.000000	76.0
38274		2	1.000000	116.0
38275		2	1.000000	106.0
38276		14	6.000000	689.0

[38277 rows x 3 columns]

Task: Create a `seaborn` pairplot of the data subset you just created. Specify the *kernel density estimator* as the kind of the plot, and make sure that you don't plot redundant plots.

Note: It will take a few minutes to run and produce a plot.

```
[ ]: # YOUR CODE HERE

# Solution:
sns.pairplot(data=df_corrs, kind = 'kde', corner=True)
```

1.6 Part 5: Analysis

1. In Part 3, you winsorized the price column and imputed missing values in bedrooms, bathrooms, and beds using column means. What are the risks of these techniques, and how could those risks affect NestIQ's price estimates?
2. In Part 5, you identified the two features most correlated with `label_price` and visualized them against the label in a pairplot. Are these features strongly or weakly correlated with the label? Are they features that should be used for our predictive machine learning problem? Additionally, Name one factor that probably influences a listing price in the real world but is not captured in this dataset. How might that gap affect NestIQ's model?
3. Inspect your data matrix. It has a few features that contain unstructured text, meaning text data that is neither numerical nor categorical. List some features that contain unstructured text that you think are valuable for our predictive machine learning problem. Are there other remaining features that you think need to be prepared for the modeling phase? Do you have any suggestions on how to prepare these features?

Record your findings in the cell below.

Sample Solutions:

1. Winsorization caps the top and bottom 1% of prices, which distorts genuinely high-end or budget listings by pulling them toward the threshold value. A luxury property gets treated like a non-luxury property, so NestIQ's model will under-predict on premium listings. Mean imputation has a different risk: it assumes missing values behave like the average, but a listing with a missing bedroom count is probably missing because it's an unusual property, not because it happens to have the average number of bedrooms. Filling with the mean pulls these listings toward the middle of the distribution and hides real patterns the model could have learned. For NestIQ, these distortions mean the model will do well on typical listings

but miss on atypical ones, like studios, lofts, or listings without bedrooms in the traditional sense.

2. In the pairplot, **accommodates** and **bedrooms** show a mild positive trend with **label_price** but no tight cluster or clear line. Just a spread of points with a slight upward tilt. One might guess that more rooms or more guests would strongly drive price, but the correlation is moderate at best. These features are still worth keeping because they carry some signal, and a model can combine weak predictors to produce useful predictions. One factor that clearly affects real-world price but is not in this dataset is seasonality. Prices vary between peak tourist seasons and off-seasons, and this dataset is a snapshot in time with no date features. Without that signal, NestIQ's model may miss on listings whose price depends on when they're booked.
3. Fellows should recognize that some text features such as **description**, **name**, **neighborhood_overview**, **host_about**, **host_name**, **host_location** contain unstructured text. Features such as **description** and **neighborhood_overview** are beneficial for the machine learning problem. The goal is for students to recognize that there are remaining fields with categorical text data. They should be able to identify fields that can benefit from one-hot encoding

1.7 Part 6. Reflection: AI Usage

1. Did you use AI tools for this lab? If yes, which ones and at what points in your work? If no, briefly explain your reasoning.
2. If you used AI, describe one specific prompt that was useful and explain why it worked. If you did not use AI, walk through one part of the lab where you had to figure something out on your own and explain how you got there.
3. How did you verify that your work was correct? What would you look for to catch a mistake, whether it came from AI or from your own reasoning?
4. What is one thing you would do differently next time, either in how you approached the lab or in how you used (or did not use) AI?

Record your findings in the cell below.